

# Week 1 - Day 5a Challenge: Recursion

In this exercise, we will mainly practice recursion. So far, you have only been briefly introduced to recursion, so we consider these problems to be relatively challenging.

□ Only start working on this exercise sheet if you have completed all material (including the supplementaries) until and including day 4.

## Exercise 1: Printing [□□]

Implement a recursive function called `print_numbers` which takes a positive integer `n` and prints all the numbers from 1 to `n`. For example:

```
printNumbers(2) == 1 2
printNumbers(3) == 1 2 3
printNumbers(5) == 1 2 3 4 5
```

Make sure to test your code properly!

```
# TODO: Your code here
def print_numbers(n):
    pass
```

## Exercise 2: Factorial [□□]

Implement a recursive function called `factorial(n)` which takes a positive integer `n` and returns the factorial of that number. For example:

```
factorial(0) == 0
factorial(1) == 1
factorial(3) == 6
factorial(5) == 120
factorial(10) == 3628800
```

```
# TODO: Your code here
def factorial(n):
    pass
```

```
# Use this code cell to test your implementation
assert factorial(0) == 1
assert factorial(1) == 1
assert factorial(2) == 2
assert factorial(3) == 6
assert factorial(5) == 120

print("Test cases passed. □")
```

## Exercise 3: Fibonacci [ ] [ ]

Remember the [Fibonacci sequence](#)? It is a series of numbers in which each number is the sum of the two preceding ones, usually starting with 0 and 1. The sequence can be defined as a recursive relation:

$$F(n) = F(n-1) + F(n-2),$$

and  $F(0) = 0$  and  $F(1) = 1$ . In a previous exercise we have already implemented this function iteratively using while loops. Now, we want to implement it recursively. Define a function `fibonacci(n)` that returns the  $n$ th number in the Fibonacci sequence. For example,

```
fibonacci(0) == 0
fibonacci(5) == 5
fibonacci(10) == 55

# TODO: Your code here

# Use this code cell to test your implementation
assert fibonacci(0) == 0
assert fibonacci(5) == 5
assert fibonacci(10) == 55

print("Test cases passed. [ ]")
```

## Exercise 4: Palindrome [ ] [ ]

Remember that a palindrome is a word that reads the same forwards and backwards. We want to implement a function that checks if a given word is a palindrome using recursion. Define the function `is_palindrome(s)` that takes a string `s` and returns `True` if `s` is a palindrome and `False` otherwise. For example:

```
palindromeCheck("racecar") == True
palindromeCheck("hello") == False
palindromeCheck("level") == True
palindromeCheck("python") == False

# TODO: Your code here

# Use this code cell to test your implementation
assert is_palindrome("racecar") == True
assert is_palindrome("hello") == False
assert is_palindrome("level") == True
assert is_palindrome("python") == False

print("Test cases passed. [ ]")
```

## Exercise 5: Counting Digits [ ] [ ]

Implement a function `count_digits(n)` recursively, which takes a positive integer `n` and returns the count of digits in `n`. For example:

```
countDigits(0) == 1
countDigits(123) == 3
countDigits(987654321) == 9

# TODO: Your code here

# Use this code cell to test your implementation
assert count_digits(0) == 1
assert count_digits(123) == 3
assert count_digits(987654321) == 9

print("Test cases passed. [ ]")
```

## Exercise 6: Sum Range [ ] [ ]

Implement a recursive function called `sum_range(a, b)` which takes two positive integers `a` and `b` and returns the sum of the numbers in between the `a` and `b` (inclusive). For example:

```
sum_range(0, 0) == 0
sum_range(1, 5) == 15
sum_range(10, 15) == 75
sum_range(5, 10) == 45

# TODO: Your code here

# Use this code cell to test your implementation
assert sum_range(0, 0) == 0
assert sum_range(1, 5) == 15
assert sum_range(10, 15) == 75
assert sum_range(5, 10) == 45

print("Test cases passed. [ ]")
```

## Exercise 7: Combinations [ ] [ ] [ ]

Consider a recursive function called `combinations`. It takes a list of numbers `lst` and a number `k` and returns all the possible combinations of elements in `lst` of size `k`.

For example:

```
combinations([1, 2, 3], 2) == [[1, 2], [1, 3], [2, 3]]
combinations([4, 5, 6], 3) == [[4, 5, 6]]
combinations([1, 2, 3, 4], 5) == []
```

```
# TODO: Your code here

# Use this code cell to test your implementation
assert sorted(combinations([1, 2, 3], 2)) == [[1, 2], [1, 3], [2, 3]]
assert sorted(combinations([4, 5, 6], 3)) == [[4, 5, 6]]
assert sorted(combinations([1, 2, 3, 4], 5)) == []

print("Test cases passed. ☑")
```

## Exercise 8: Subset [ ] [ ] [ ]

Consider a recursive function called `subsets(nums)` which takes a list of integers `nums` and returns a list containing all possible subsets of `nums`. For example:

```
generateSubsets([1, 2, 3]) == [[], [1], [2], [3], [1, 2], [1, 3], [2, 3], [1, 2, 3]]
generateSubsets([4, 5]) == [[], [4], [5], [4, 5]]
generateSubsets([6]) == [[], [6]]
```

*Note: A subset is a collection of elements from the original set, and it can include elements in any order.*

```
# TODO: Your code here

assert sorted(subsets([1, 2, 3])) == [[], [1], [1, 2], [1, 2, 3], [1, 3], [2], [2, 3], [3]]
assert sorted(subsets([4, 5])) == [[], [4], [4, 5], [5]]
assert sorted(subsets([6])) == [[], [6]]

print("Test cases passed. ☑")
```

## Exercise 9: Nested List Sum [ ] [ ] [ ]

Implement a recursive function called `nested_list_sum(lst)` which takes a nested list `lst`, which can contain either integers or nested lists and returns the sum of all the integers in the nested list. For example:

```
nested_list_sum([1, 2, 3, [4, 5]]) == 15
nested_list_sum([[1, 2], [3, [4, 5]]]) == 15
nested_list_sum([1, [2, [3, [4, [5]]]]) == 15

# TODO: Your code here

# Use this code cell to test your implementation
assert nested_list_sum([1, 2, 3, [4, 5]]) == 15
assert nested_list_sum([[1, 2], [3, [4, 5]]]) == 15
assert nested_list_sum([1, [2, [3, [4, [5]]]]) == 15

print("Test cases passed. ☑")
```

## Exercise 10: Permutations [ ] [ ] [ ]

Implement a recursive function called `permutations(lst)` which takes a list of numbers and returns all **unique permutations** of the list in any order. For example:

```
permutations([1, 1]) == [[1, 1]]
permutations([1, 1, 2]) == [[1, 1, 2], [1, 2, 1], [2, 1, 1]]
permutations([1, 2, 3]) == [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2],
[3, 2, 1]]
```

*Note: A unique permutation refers to a distinct arrangement of the elements in a given list, where each arrangement is different from all others.*

```
# TODO: Your code here

# Use this code cell to test your implementation
assert sorted(permutations([1, 1])) == [[1, 1]]
assert sorted(permutations([1,1,2])) == [[1,1,2], [1,2,1], [2,1,1]]
assert sorted(permutations([1,2,3])) == [[1,2,3],[1,3,2],[2,1,3],
[2,3,1],[3,1,2],[3,2,1]]

print("Test cases passed. ☐")
```

## Exercise 11: Tower of Hanoi [□□□]

You may have heard of the game **Tower of Hanoi**. As seen in the picture below, the goal of the game is to move all the disks from the leftmost peg to the rightmost peg. The middle peg can be used as a helper peg to temporarily hold the disks.

There are only two simple rules:

1. You can only move one disk at a time.
2. You can only place a smaller disk on top of a larger disk.

The puzzle starts with `n` disks on the leftmost peg, and the disks are sorted from smallest to largest, with the largest disk on the bottom. Your task is to write a **recursive function** called `hanoi(n, source, helper, target)` that prints out the steps to solve the Tower of Hanoi puzzle. For example:

```
hanoi(1, "A", "B", "C") # ---> prints "Move disk 1 from peg A to peg
C"
hanoi(2, "A", "B", "C") # ---> prints "Move disk 1 from peg A to peg
B"
                                #                "Move disk 2 from peg A to peg
C"
                                #                "Move disk 1 from peg B to peg
C"
```

```

hanoi(3, "A", "B", "C") # ---> prints "Move disk 1 from peg A to peg
C"
                                #           "Move disk 2 from peg A to peg
B"                                #           "Move disk 1 from peg C to peg
B"                                #           "Move disk 3 from peg A to peg
C"                                #           "Move disk 1 from peg B to peg
A"                                #           "Move disk 2 from peg B to peg
C"                                #           "Move disk 1 from peg A to peg
C"

```

**Hint:** Think about what the base case and recursive case would be; the recursive case should reduce the problem to a smaller version of itself (in this problem, a Tower of Hanoi with fewer disks).

```

# TODO: Your code here
def hanoi(n, source, helper, target):
    # NOTE: You don't need to return anything from the function; just
    print the moves
    pass

```